DESIGN

The purpose of this document is to outline the architectural design and system components of the proposed **IoT-Based Smart Home System**. This system aims to provide a secure, efficient, and user-friendly home monitoring environment by integrating environmental sensing, automated resource management, and AI-supported security features.

The design prioritizes an **"Edge-First"** approach. Critical triggers and initial processing are performed locally on the device to ensure reliability and privacy, while cloud services are leveraged for heavy processing (AI Face Matching), data persistence, and remote accessibility.

The high-level system architecture consists of the following key nodes:

- **Edge Node (Raspberry Pi 5 Edge):** Runs the local Sensor Driver Service and Rule Engine. It communicates with the cloud over Wi-Fi using secure protocols (HTTPS/MQTT). It is responsible for "Graceful Degradation," ensuring local alarms function even without internet connectivity.
- **Cloud Node (Cloud Backend Server):** Hosts the REST API and Database Server. It acts as the orchestrator for state management and relays push notifications to the external provider when critical events are detected.
- **Client Nodes:** The Web Dashboard and Mobile Application fetch data from the cloud node via API requests, allowing users to view real-time status and historical logs.

1. Design of Home Controller

The hardware architecture is centered around a **Raspberry Pi 5** single-board computer, acting as the main home controller and edge processing unit. The system integrates various sensors and actuators through specific digital and analog interfaces to minimize latency.

- **Main Controller (Raspberry Pi 5):** Selected for its high processing power, enabling local sensor management and preliminary event detection.
- **Vision Module:** A Raspberry Pi Camera Module connected via the **CSI (Camera Serial Interface)** port to provide high-bandwidth video data for the surveillance system.
- **Environmental Sensors:** The **DHT11** (Temperature & Humidity) and **MQ-2** (Gas & Smoke) sensors are connected via GPIO pins to facilitate climate tracking and emergency detection.
- **Resource Monitoring:** A Load Cell paired with an **HX711 Amplifier Module** connects to digital pins to precisely measure weight for the "Automated Pet Resource Monitoring" feature.
- **Storage:** A high-speed microSD Card (64GB) is utilized for the OS, local logs, and image buffering during offline operation (Graceful Degradation).

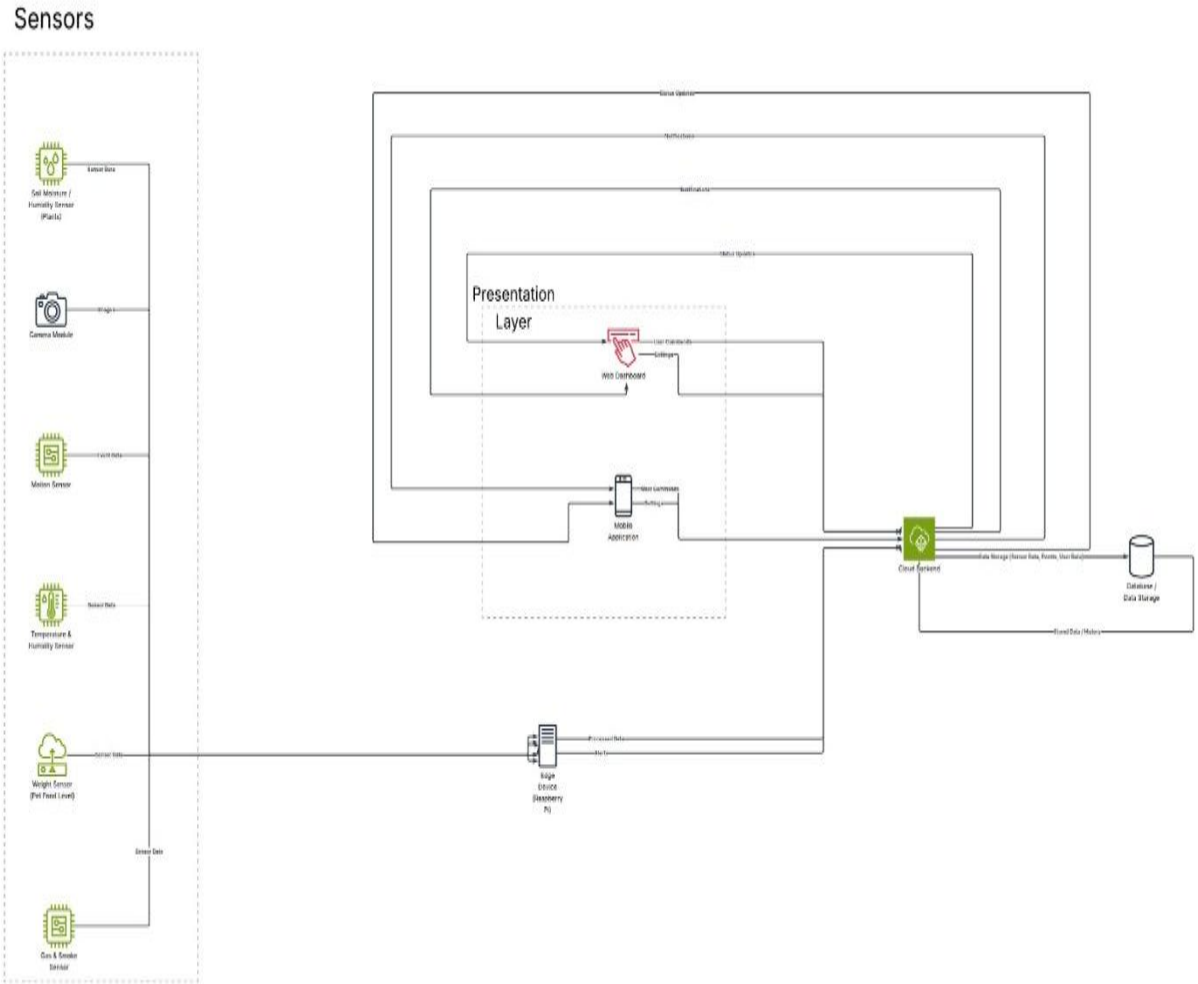

Figure 1 Design of Home Controller

2. Software Architecture Design

The software architecture follows a modular **4-Tier Layered Architecture** pattern to ensure separation of concerns, scalability, and maintainability.

2.1 Layer Decomposition

Layer 1: Presentation (User Interfaces)

Figure 2 Notification

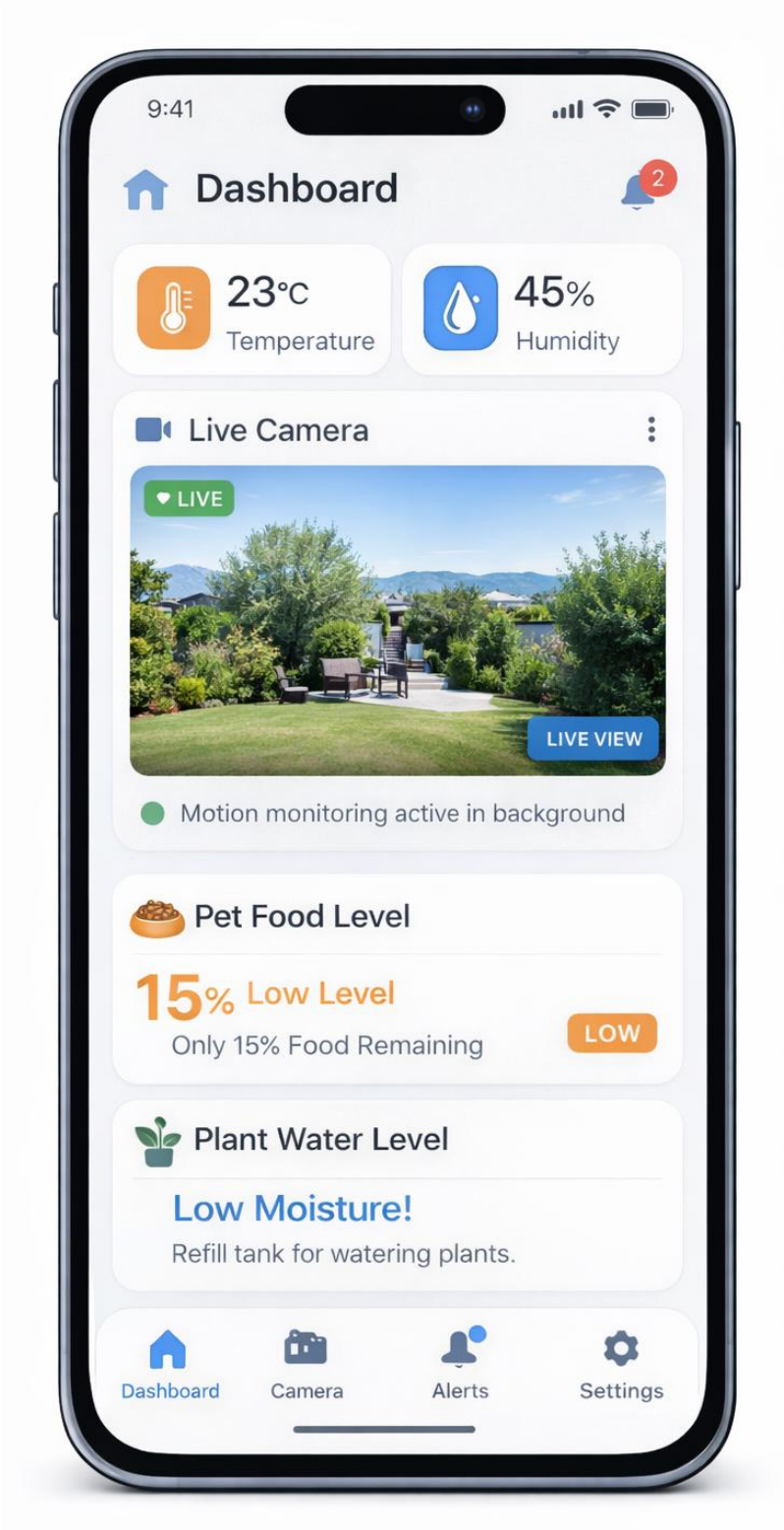

Figure 3 Mobil Application

This layer handles user interactions via the Mobile Application and Web Dashboard. It communicates solely through API calls, ensuring a stateless and responsive user experience. It allows users to view dashboards, receive notifications, and send manual commands (e.g., "Open Camera").

Figure 4 Web Application

Layer 2: Security

Serving as the gateway, this layer manages **Authentication & Authorization**. It ensures that all communication is encrypted using **HTTPS/TLS** protocols. It verifies user identities before any command reaches the business logic.

Layer 3: Business Logic (Core Modules)

This is the core of the system where data processing occurs. To maintain a clean architecture and clear separation of duties, the functional requirements are grouped into four distinct handler modules:

1. **Climate Monitoring Service:** Handles periodic reading and logging of temperature and humidity data.
2. **Pet Resource Manager:** Manages the monitoring of food/water bowls and triggers alerts for low levels.
3. **Emergency Response Manager:** Dedicated to interrupt-based critical tasks like "Detecting Fire and Flood" to ensure immediate local feedback.
4. **Surveillance & Intrusion Analysis Engine:** A comprehensive security engine that handles the single-loop logic for motion capture, cloud-based face recognition, and the "Frequency Analysis" of unknown visitors.

Layer 4: Data & Integration

This layer manages persistence via the **Cloud Database** (storing user profiles, logs, and safe/non-safe face vectors) and handles third-party integrations, such as the External Push Notification Provider.

3. Detailed Module Design & Sequence Flows

This section details the internal algorithms, logic flows, and interaction sequences for the key functions defined in the Business Logic Layer.

3.1 Climate Monitoring Service

Addresses Requirements: `[Req. 2.1.1]`

Since continuous data streaming places unnecessary load on the network, this module utilizes a **"Polling Mechanism"**.

- **Logic:** The system wakes up at a configurable interval (default: 30s), reads the DHT11 sensor via GPIO, validates the data range, and transmits the packaged JSON payload to the Cloud Backend.
- **Sequence:** The interaction flow for periodic climate tracking is illustrated below.

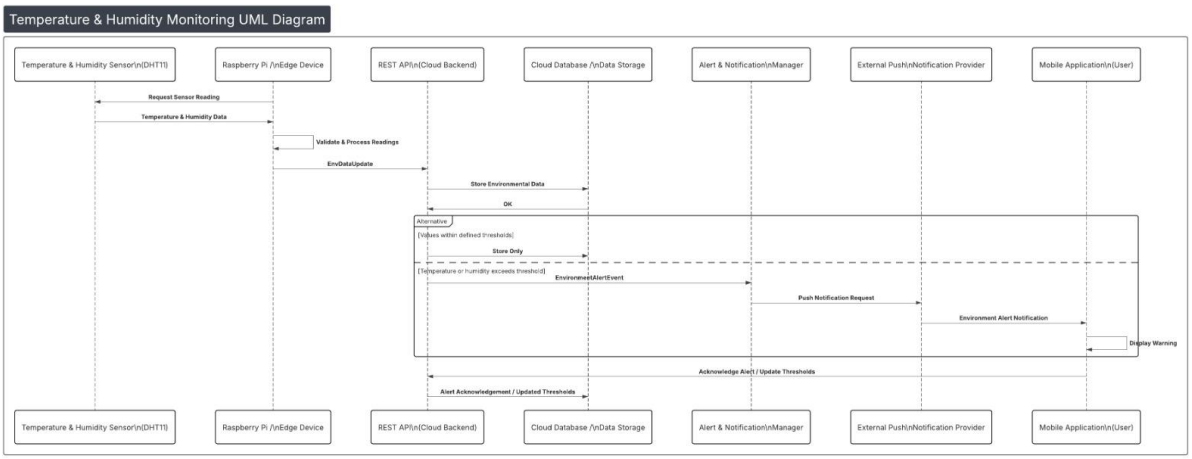

Figure 5 Temperature & Humidity Sequence Diagram

3.2 Pet Resource Manager

Addresses Requirements: `[Req. 2.1.4]`

This module focuses on ensuring pet safety by tracking resource availability.

- **Logic:** The system reads the Load Cell (via HX711). The analog weight is converted to a percentage (0-100%). If `Weight < Threshold (10%)`, a **"Low Resource Event"** is flagged. It includes a cooldown timer to prevent notification fatigue (e.g., only one alert every 4 hours).
- **Sequence:** The flow for weight measurement and alert triggering is shown below.

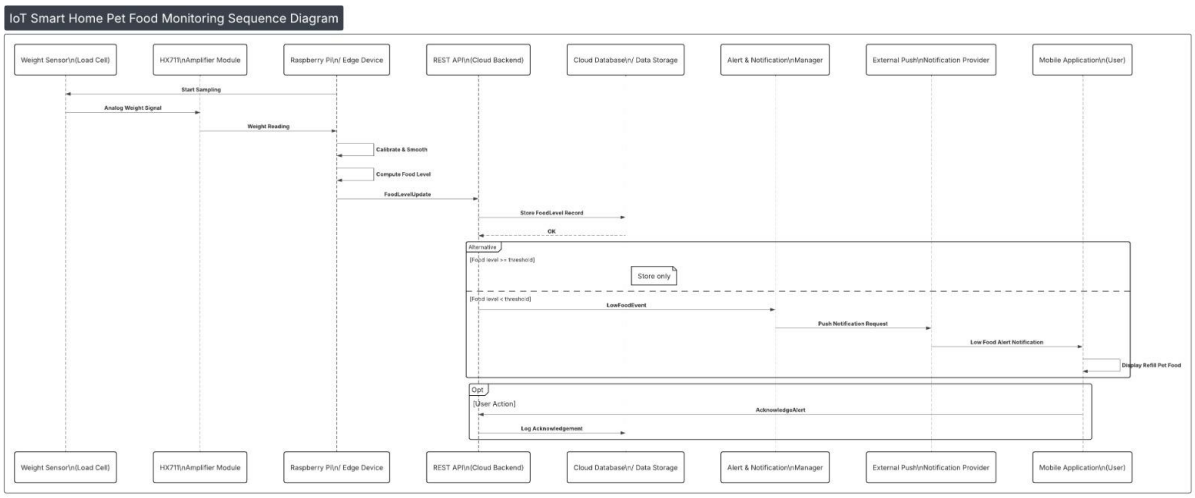

Figure 6 Pet Food Monitoring Sequence Diagram

3.3 Emergency Response Manager

Addresses Requirements: `[Req. 2.1.2]` & `[Req. 3.2]`

Unlike the polling mechanism, emergency sensors operate on an **Interrupt-based Architecture** to ensure immediate response.

- **Logic:** When the Gas/Smoke sensor triggers, the main loop is interrupted. The Edge Node immediately triggers the local buzzer (Graceful Degradation) while simultaneously sending a high-priority alert to the Cloud Backend for push notifications.
- **Sequence:** The flow from detection to user notification is shown below.

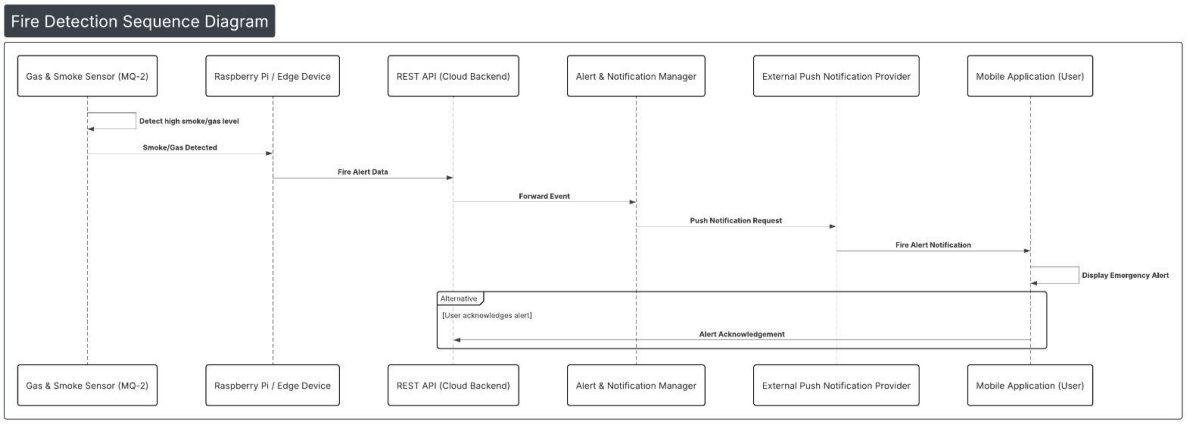

Figure 7 Fire Detection Sequence Diagram

3.4 Surveillance & Intrusion Analysis Engine

Addresses Requirements: `[Req. 2.2.1]`, `[Req. 2.2.2]`, `[Req. 2.4.3]`

The security system operates on a **Continuous Single-Loop Mechanism** to avoid complex multi-threading issues and ensure stability.

- **Logic Flow:**

1. **Sequential Check:** The system first checks for a "Manual Stream Request" from the app. If none, it checks the PIR Motion Sensor.
2. **Capture & Filter:** If motion is detected, images are captured. A local lightweight model checks for human presence.
3. **Cloud Identification:** Images are sent to the cloud. The AI compares the face against the "Safe User" database.
4. **Frequency Analysis:** If unauthorized, the system checks visit history (Frequency Analysis) and notifies the user with a visit count, asking for verification.

- **Sequence:** The complete flow from motion detection to AI classification and user notification is detailed below.

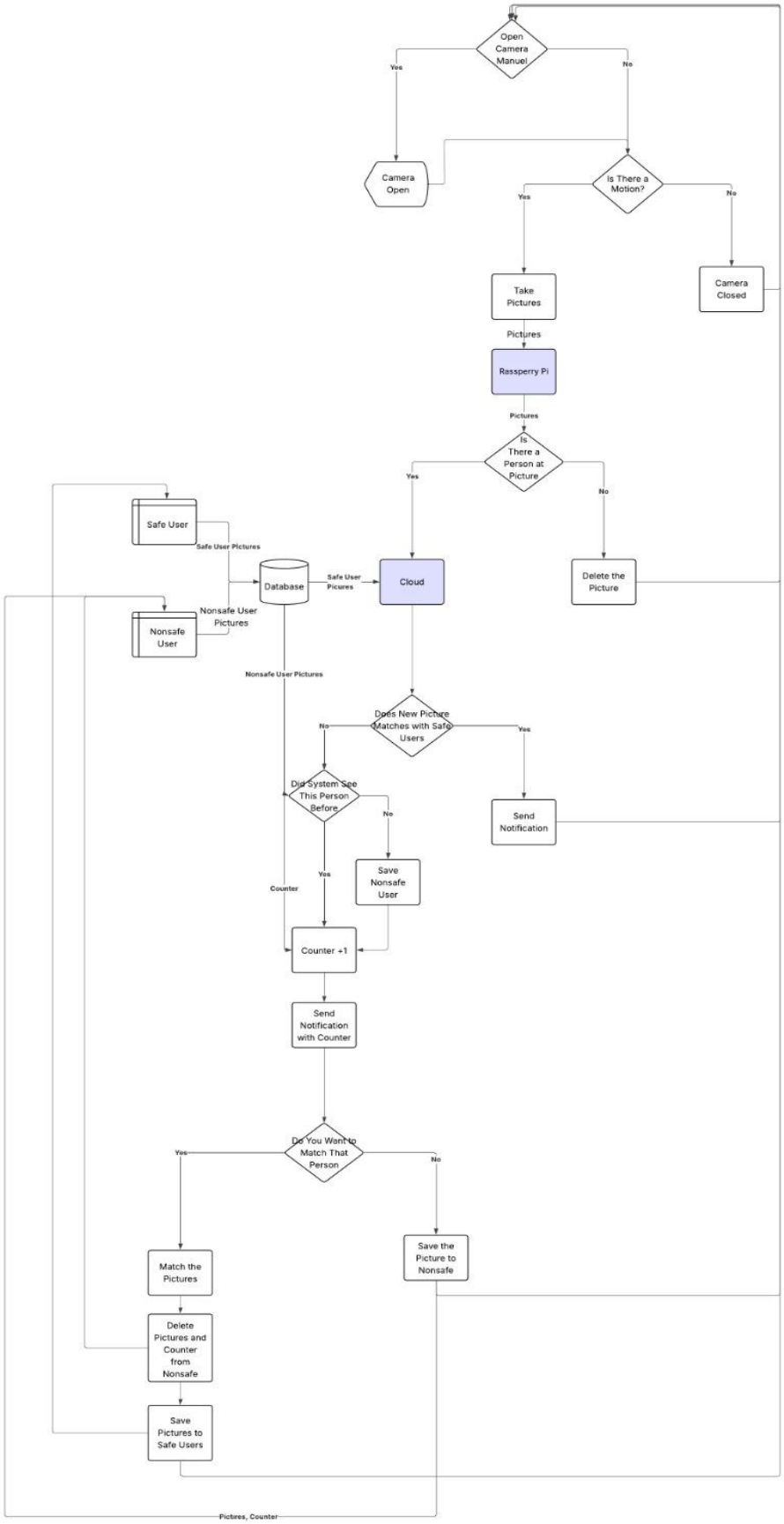

Figure 8 Classifying Authorized/Unauthorized Persons Sequence Diagram